

Sistemi Operativi (Modulo II)

Capitolo 7— File System

Luca Argentino

A.A. 2025–2026

Indice

7	File System	2
7.1	Visione utente	2
7.1.1	Introduzione	2
7.1.2	Attributi dei file	3
7.1.3	Tipi di file	4
7.1.4	Struttura dei file	5
7.1.5	Metodi di accesso	6
7.1.6	Operazioni sui file	6
7.1.7	Strutture di directory	7
7.1.8	L'i-node: Struttura e Puntamento	9
7.1.9	Riepilogo Comparativo Link	10
7.1.10	Semantica della coerenza	11
7.2	Visione implementatore	12
7.2.1	Organizzazione del disco	12
7.2.2	Allocazione dei blocchi	14
7.2.3	Gestione dello spazio libero	22
7.2.4	Implementazione delle directory	25
7.2.5	Performance del file system	28
7.2.6	Tecniche per garantire la coerenza	29
7.2.7	Esempi di file system reali	31
7.3	Riepilogo comparativo	34

Capitolo 7

File System

Il capitolo è organizzato in due grandi sezioni: prima vediamo il file system dal punto di vista dell'**utente** (cosa sono file e directory, come si accede, come si organizzano), poi dal punto di vista dell'**implementatore** (come il sistema operativo gestisce il disco, alloca i blocchi, garantisce la coerenza).

7.1 Visione utente

7.1.1 Introduzione

I computer possono utilizzare diversi media per registrare in modo permanente le informazioni: dischi rigidi, SSD, nastri, dischi ottici, chiavette USB, ... Ognuno di questi media ha caratteristiche fisiche molto diverse tra loro.

Definizione – File System

Il **file system** è il componente del sistema operativo che *astrae* la complessità di utilizzo dei diversi media di memorizzazione, proponendo un'interfaccia comune, efficiente e conveniente da usare.

Dal punto di vista dell'utente, un file system è composto da due elementi fondamentali:

- **File:** unità logica di memorizzazione, entità atomica di assegnazione/gestione della memoria secondaria.

È una collezione di informazioni correlate che fornisce una vista logica uniforme.

- **Directory:** servono per organizzare i file e fornire informazioni su di essi.

E' come un catalogo di informazioni

FILE

7.1.2 Attributi dei file

Ogni file possiede un insieme di **attributi** che ne descrivono le proprietà. I principali sono:

Attributo	Descrizione
Nome	Stringa di caratteri per identificare il file. Alcuni sistemi distinguono maiuscolo/minuscolo, altri no.
Tipo	Necessario in alcuni sistemi per identificare il tipo di file (es. eseguibile, testo, ...).
Locazione e dimensione	Informazioni sul posizionamento in memoria secondaria e sulla dimensione in byte.
Data e ora	Istanti di creazione, ultima modifica e ultimo accesso.
Proprietà	Utente, gruppo, usati per accounting e autorizzazione.
Attributi di protezione	Chi è autorizzato a leggere, scrivere, eseguire.
Altri attributi	Flag di sistema, archivio, hidden, lock, ...

```

Terminale Bash
$ ls -l documento.txt
-rw-r--r-- 1 luca studenti 1024 9 apr 11:17 documento.txt

$ stat documento.txt
  File: documento.txt
  Size: 1024          Blocks: 8          IO Block: 4096   regular file
Device: 802h/2050d Inode: 123456        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   luca)   Gid: (
      1000/studenti)
Access: 2026-04-09 11:17:38.000000000 +0200
Modify: 2026-04-09 11:10:00.000000000 +0200
Change: 2026-04-09 11:10:00.000000000 +0200
 Birth: -

```

Nota – Analisi dell'output

I comandi eseguiti mostrano come il sistema operativo (in questo caso Linux) esponga all'utente gli attributi associati a `documento.txt`:

- **ls -l**: fornisce una vista sintetica dei metadati. Mostra il tipo e gli attributi di protezione (`-rw-r--r--`), il numero di link (1), le informazioni di proprietà indicando l'utente (`luca`) e il gruppo (`studenti`), la dimensione in byte (1024), e la data dell'ultima modifica.
- **stat**: interroga l'astrazione del file system per restituire i metadati completi. Oltre a confermare i dati precedenti, rivela il tipo di file (**regular file**), i dettagli sulla locazione (tramite l'indice logico **Inode**: 123456 e il numero di **Blocks** allocati). Inoltre, distingue con altissima precisione i tre principali timestamp: accesso (**Access**), modifica del contenuto (**Modify**) e cambio degli attributi o metadati (**Change**).

7.1.3 Tipi di file

I file si possono classificare in base alla struttura interna o al contenuto.

Classificazione per struttura interna

- **Senza formato** (*unstructured*): semplice sequenza di byte, es. file di testo.
- **Con formato**: file di record, database, eseguibili (*a.out*), ...

Classificazione per contenuto

- ASCII / binario (visualizzabile o meno, 7 o 8 bit).
- Sorgente, oggetto, eseguibile (oggetto *attivo*).

Tecniche di identificazione del tipo

I sistemi operativi usano tre tecniche principali per riconoscere il tipo di un file:

1. **Meccanismo delle estensioni**: il tipo è codificato nella parte finale del nome (es. *.exe*, *.pdf*).
2. **Attributo “tipo”** nella directory entry: il tipo viene memorizzato come metadato associato al file.
3. **Magic number**: i primi byte del file contengono una firma identificativa del formato (usato da UNIX/Linux).

Nota – Approcci per S.O.

- **MS-DOS/FreeDOS**: nomi 8+3(nome+estensione) caratteri, riconosce *.COM*, *.EXE*, *.BAT*.
- **Windows 9x/NT**: nomi a lunghezza variabile, associazione estensione ↔ programma.
- **Mac OS**: programma creatore come attributo del file.
- **Unix/Linux**: magic number + estensione + euristica.

Tipi speciali in UNIX

Nei file system UNIX esistono diverse categorie di file:

- **File regolari**: sequenze di byte, il tipo "normale" di file.
- **Directory**: file di sistema per mantenere la struttura del file system.
- **File speciali a blocchi**: modellano dispositivi di I/O come i dischi. Utilizzati per accedere alla memoria secondaria
- **File speciali a caratteri**: modellano dispositivi seriali (terminali, stampanti, reti).
- **Altri file speciali**: ad esempio le *pipe* (FIFO).

Attenzione

I file speciali (blocchi, caratteri, pipe) **NON sono veri file**: sono astrazioni del sistema operativo che appaiono nel file system per uniformità di interfaccia, ma non contengono dati su disco.

7.1.4 Struttura dei file

I file possono essere strutturati in tre modi principali:

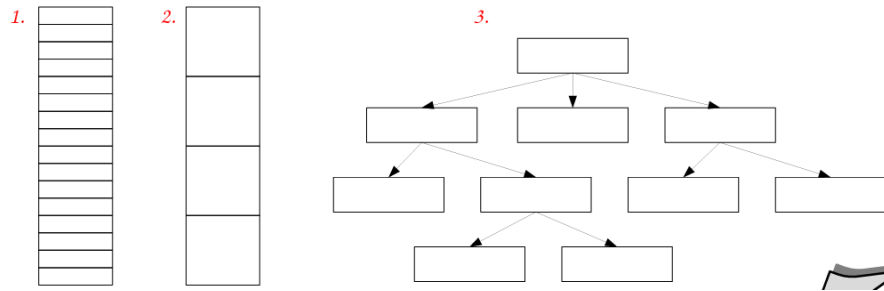


Figura 7.1: Tre modi di strutturare un file: sequenza di byte, sequenza di record logici, file ad albero indicizzato.

1. **Sequenza di byte:** struttura minimale. Il file è una stringa di byte senza semantica aggiuntiva. È la scelta di UNIX e MS-DOS: massima flessibilità per le applicazioni.
2. **Sequenza di record logici:** il file è diviso in blocchi logici (record) di dimensione fissa o variabile.
3. **File indicizzati:** struttura ad albero, tipica dei database. Ogni record è accessibile tramite una chiave.

Nota – Trade-off sulla struttura

Supportare più formati richiede più codice di sistema ma garantisce efficienza e non duplicazione della logica. Supportare meno formati produce codice di sistema più snello ma sposta la complessità nelle applicazioni.

7.1.5 Metodi di accesso

Definizione – Metodi di accesso ai file

- **Sequenziale:** `read/write` in ordine dall’inizio alla fine del file.
- **Ad accesso diretto** (*random access*): `read pos/write pos` oppure operazione `seek(offset)` per saltare ad una posizione arbitraria.
- **Indicizzato:** `read key/write key`, tipico dei database. Richiede la gestione di un indice chiave → posizione.

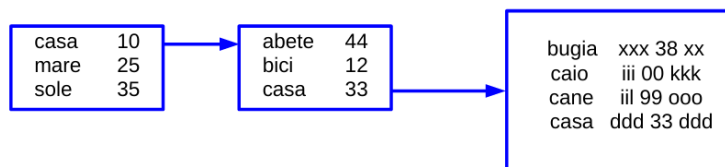


Figura 7.2: Esempio di indice: tabella di corrispondenza chiave–posizione. L’indice può essere tenuto in memoria (efficiente ma dispendioso) o su disco.

7.1.6 Operazioni sui file

Le operazioni fondamentali sui file sono:

creazione, apertura/chiusura, lettura/scrittura/append, posizionamento, cancellazione, troncamento, lettura/scrittura degli attributi.

Nota

Tutte queste operazioni le abbiamo viste come system call offerte da Linux

L’API relativa alle operazioni su file si basa sul paradigma **open/close**:

- un file deve essere “aperto” prima di effettuare operazioni
- “chiuso” al termine.

L’astrazione apertura/chiusura dei file è utile per:

- Mantenere le strutture dati di accesso al file.
- Controllare le modalità di accesso e gestire gli accessi concorrenti.
- Definire un **descrittore** (file descriptor) per le operazioni successive di accesso ai dati.

DIRECTORY

Definizione – Directory

Una **directory** è un file speciale che fornisce un'astrazione per un insieme di file, permettendo di organizzarli e recuperarne le informazioni.

In molti sistemi, le directory *sono* file (speciali).

Le **operazioni** sulle directory includono:

creazione, cancellazione, apertura/chiusura, lettura, rinominazione, link/unlink.

7.1.7 Strutture di directory

- **Directory a livello singolo e a due livelli.** Nei sistemi più semplici tutte le entry stanno in un unico livello, o al massimo in due (un livello per utente). Non scalano bene.
- **Directory strutturata ad albero.** La struttura più comune: ogni file ha un percorso unico dalla radice (*path*). Ogni directory può contenere file e sottodirectory.

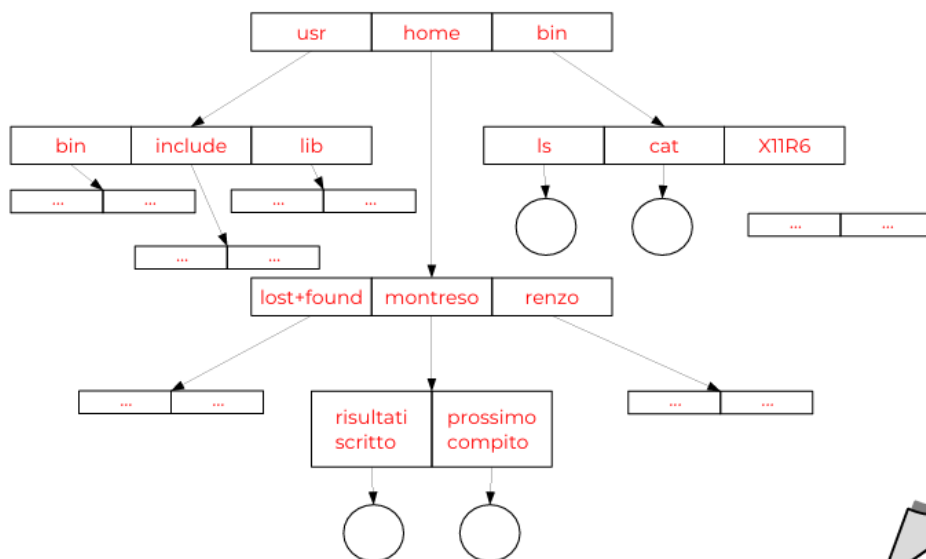


Figura 7.3: Struttura a directory organizzata come albero. Ogni file appartiene ad una e una sola directory.

- **Directory strutturata a grafo aciclico (DAG).**

Nella struttura ad albero ogni file è contenuto in una directory univoca.

Con i grafi diretti aciclici (DAG) un file può essere contenuto in due o più directory mantenendo un'unica copia fisica: ogni modifica è visibile da entrambe le directory.

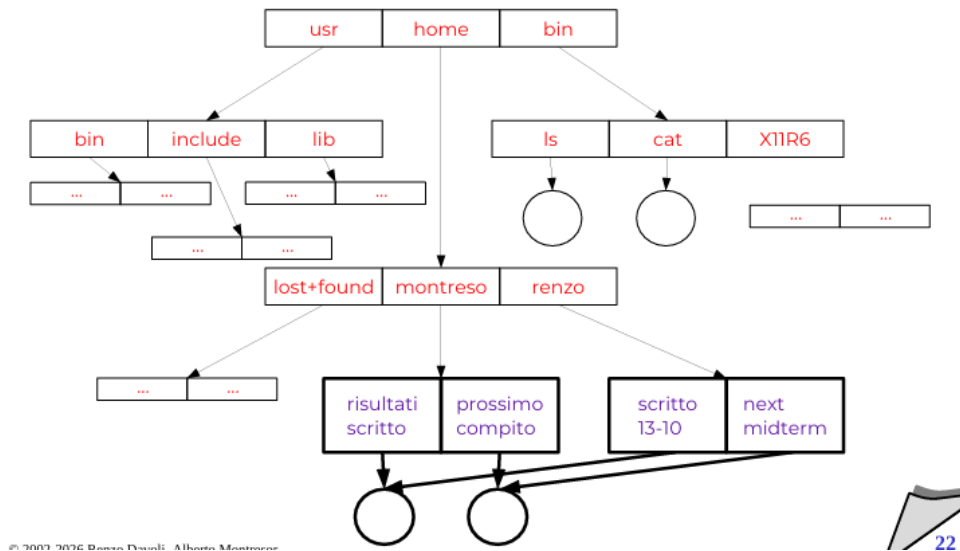


Figura 7.4: Directory strutturata a grafo aciclico (DAG): il file **scritto** è accessibile da due percorsi distinti.

Implementazione dei DAG: link

Esistono due implementazioni per la struttura DAG:

- **Link simbolici** (*soft link*): si crea una directory entry speciale contenente il percorso assoluto del file target. Quando si accede al file si segue il percorso registrato. È possibile avere link pendenti (*dangling links*) se il target viene cancellato.
- **Hard link**: le informazioni del file compaiono, identiche, in entrambe le directory. Non c'è distinzione tra originale e copia. Richiede i-node con **contatore di riferimenti**: il file viene eliminato solo quando il contatore scende a zero.

• Implementazione

- E' necessario utilizzare la tecnica degli i-node
- gli i-node devono contenere un contatore di riferimenti

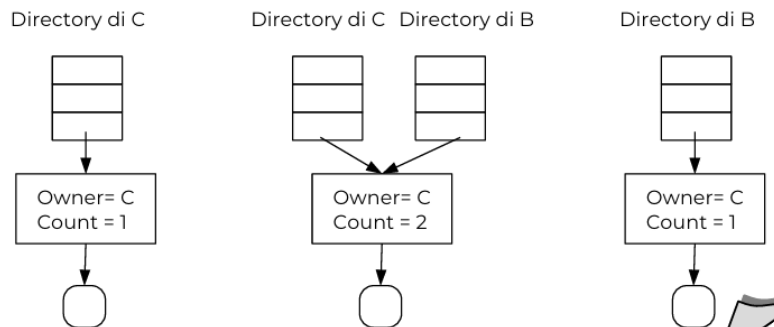


Figura 7.5: Implementazione degli hard link con contatore di riferimenti nell'i-node. A sinistra: il file ha un solo riferimento (count = 1). Al centro: dopo il link da B, il count sale a 2.

Nota

Gli hard link non possono essere creati tra file system diversi (perché gli i-node sono locali al file system). I link simbolici invece funzionano anche tra file system differenti. MS-DOS non supporta i DAG.

7.1.8 L'i-node: Struttura e Puntamento

Definizione – i-node (index node)

L'i-node è la struttura dati fondamentale nei file system UNIX-like. Ogni file è identificato univocamente da un numero di i-node, che contiene tutti i metadati del file (tranne il nome).

- **Attributi:** Dimensione, permessi, UID/GID e il **contatore di riferimenti** (fondamentale per gli hard link).
- **Puntatori ai dati:** UNIX usa uno schema misto per gestire file di ogni dimensione^a:
 - Puntatori Diretti
 - Puntatori Indiretti

^aVedi pag 20 per spiegazione dettagliata

Nota – Efficienza

Lo schema misto è ottimale perché la maggior parte dei file è piccola (pochi KB) e viene acceduta con un solo accesso al disco, mentre i file enormi pagano il costo dell'indirizzamento solo quando necessario.

• **Le Directory:** È l'intera tabella che vedi sulla sinistra, proprio sotto l'intestazione in corsivo "Directory: name inode n.". Questa tabella nel suo complesso rappresenta il "catalogo" di cui parlavamo prima.

• **Le Directory Entry:** Sono le singole righe all'interno di quella tabella. Nello specifico, in questa directory ci sono tre *directory entry* visibili:

1. La prima riga, che contiene il nome **file** e il puntatore al suo i-node.
2. La seconda riga, che contiene il nome **hlink** e il puntatore al suo i-node.
3. La terza riga, che contiene il nome **slink** e il puntatore al suo i-node.

Come puoi notare, essendo questo l'approccio tipico dei sistemi UNIX, ogni singola *entry* (riga) è molto snella e si limita ad associare un nome a un i-node, demandando tutte le altre informazioni (gli attributi) all'i-node stesso sulla destra.

7.1.9 Riepilogo Comparativo Link

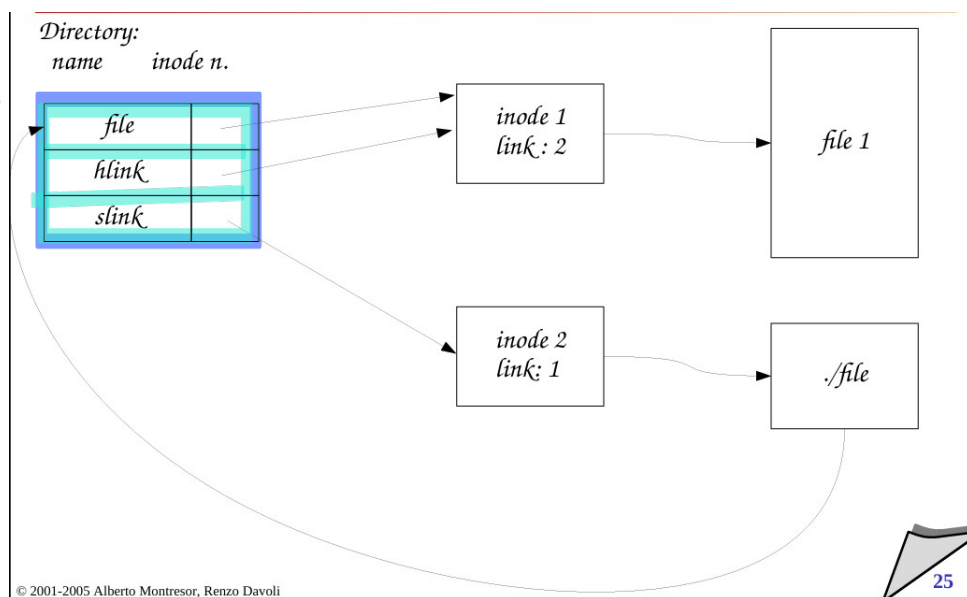


Figura 7.6: Rappresentazione grafica dei link. Il file originale (**file**) e l'hard link (**hlink**) condividono lo stesso **inode 1** (count = 2). Il soft link (**slink**) usa l'**inode 2** per memorizzare il percorso logico che punta al nome **file**.

Attenzione – Cosa succede se si elimina il file originale?

Osservando lo schema in Figura 7.6, possiamo capire esattamente cosa accade "sotto il cofano" se diamo il comando per cancellare la voce **file** (tramite ad esempio la sys call **unlink**):

- **Impatto sull'Hard Link (hlink):** Cancellare **file** rimuove solo quel nome dalla directory e abbassa il contatore dell'**inode 1** (da **link: 2** a **link: 1**). Dato che il contatore *non* è sceso a zero, i dati fisici (**file 1**) non vengono toccati. **Risultato:** **hlink** continua a funzionare perfettamente e i dati sono salvi.
- **Impatto sul Soft Link (slink):** L'**inode 2** di questo link non viene modificato, ma il suo contenuto testuale (**./file**) punta ora a un nome che non esiste più nella directory. **Risultato:** Il soft link si "rompe" diventando un link pendente (*dangling link*). Qualsiasi tentativo di aprirlo darà errore.

Caratteristica	Hard Link	Soft Link (Simbolico)
Punta a...	All'i-node fisico.	Al percorso (path) testuale.
Oltre FS	No (gli i-node sono locali).	Sì (usa i path).
Originale eliminato	Il file resta finché count > 0.	Il link si "rompe" (dangling).
Identità	È il file stesso.	È un file speciale separato.

Tabella 7.1: Confronto riassuntivo tra tipologie di collegamenti.

7.1.10 Semantica della coerenza

In un sistema operativo multitasking (o in un sistema distribuito), più processi possono accedere allo stesso file simultaneamente. La semantica della coerenza definisce come e quando le modifiche effettuate da un processo diventano visibili agli altri.

Semantica UNIX (Visibilità Immediata)

Nei file system UNIX, le modifiche al contenuto di un file aperto vengono rese visibili agli altri processi *immediatamente*. Esistono due modalità di condivisione per l'accesso ai file:

- **Condivisione del puntatore** (Offset condiviso): tipica dei processi creati tramite la system call `fork()`, in cui padre e figlio condividono lo stesso indicatore di posizione (offset). Avanzando la lettura o la scrittura, il puntatore si sposta per entrambi.
- **Puntatori distinti**: processi indipendenti che aprono lo stesso file hanno ciascuno il proprio offset separato, potendo leggere o scrivere in posizioni diverse dello stesso file.

Nota

Nota: Semantica delle Sessioni (es. AFS)

Non tutti i sistemi adottano la *semantica immediata* di UNIX. AFS (*Andrew File System*), essendo un file system distribuito su rete, utilizza la **semantica delle sessioni**.

- **Come funziona**: quando si lavora su un file, le modifiche vengono effettuate solo su una copia locale. Tali modifiche diventano visibili agli altri utenti sulla rete solo al momento della chiusura del file (quando la "sessione" termina e il file viene inviato al server).
- **Perché si usa**: applicare la semantica immediata in un file system distribuito renderebbe il sistema estremamente lento. Sincronizzare ogni singola operazione di scrittura in tempo reale attraverso la rete genererebbe un traffico e una latenza insostenibili per le prestazioni del sistema.

7.2 Visione implementatore

Panoramica dei problemi implementativi

Dal punto di vista dell'implementatore, realizzare un file system richiede di affrontare i seguenti problemi:

1. **Organizzazione del disco:** come è strutturato fisicamente (partizioni, superblock, ...).
2. **Allocazione dei blocchi:** come si scelgono e collegano i blocchi che compongono un file.
3. **Gestione dello spazio libero:** come si tiene traccia dei blocchi non utilizzati.
4. **Implementazione delle directory:** come si memorizzano le directory entry.
5. **Performance:** tecniche di caching e ottimizzazione.
6. **Coerenza:** come garantire la consistenza dopo un crash.

7.2.1 Organizzazione del disco

Struttura del disco

Un disco può essere diviso in una o più **partizioni**, porzioni indipendenti che possono ospitare file system distinti.

Il **primo settore** del disco è il *Master Boot Record* (MBR) oppure, nei sistemi moderni, la *GUID Partition Table* (GPT).



Figura 7.7: Struttura del disco: MBR (o GPT) seguito dalle partizioni. L'MBR contiene la partition table e viene letto all'avvio.

Definizione – MBR – Master Boot Record

Il primo settore del disco. Contiene:

- Il codice di boot (caricato ed eseguito dal BIOS).
- La **partition table**: posizione e dimensione di ciascuna partizione (1,2,3 come vediamo in 7.7).
- L'indicazione della **partizione attiva** (da cui fare il boot).

Struttura di una partizione

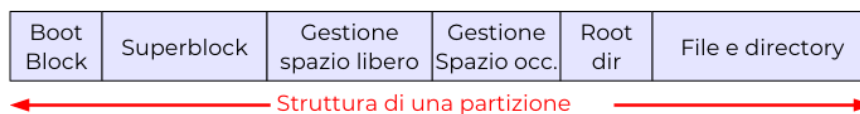


Figura 7.8: Struttura interna di una partizione logica: boot block, superblock, gestione dello spazio libero/occupato, root directory, file e directory.

Area	Contenuto
Boot block	Caricato dall'MBR; a sua volta carica il sistema operativo, presente anche nelle partizioni non attive.
Superblock	Informazioni sul tipo di file system e sui suoi parametri fondamentali (dimensione blocchi, dov'è la tabella spazi occupati, numero di i-node, ...).
Gestione spazio libero	Struttura dati che tiene traccia dei blocchi liberi (bit-map o lista).
Gestione spazio occupato	Informazioni sui file presenti (es. i-node table). Non presente in tutti i file system.
Root dir	La directory radice del file system.
File e directory	Il resto della partizione contiene i dati effettivi.

Perché il codice di boot è presente sia nell'MBR che nelle singole partizioni?

Il processo di avvio di un computer funziona come una *staffetta a più stadi*, in cui ogni componente ha un livello di complessità maggiore e passa il controllo a quello successivo:

- **MBR (Master Boot Record) – Il "Centralinista":** Il BIOS è un software di base che non "comprende" i file system complessi (come NTFS o ext4). Sa solo leggere il primissimo settore fisico del disco, ovvero l'MBR. Essendo molto piccolo, il codice di boot dell'MBR si limita a leggere la *Partition Table*, individuare la partizione marcata come "attiva" e cedere il controllo al primissimo settore di quella specifica partizione.
- **Il Boot Block della Partizione – Lo "Specialista":** È il primo settore della singola partizione. A differenza dell'MBR, il codice contenuto nel Boot Block "conosce" lo specifico file system con cui è stata formattata la partizione. Grazie a questa conoscenza, è in grado di navigare nel file system per cercare e caricare in RAM i file effettivi del sistema operativo.

7.2.2 Allocazione dei blocchi

L'hardware e il driver del disco espongono il disco come un insieme di **blocchi dati** di dimensione fissa.

Il problema dell'allocazione è: *come scegliere i blocchi da usare per un file e come collegarli tra loro?*

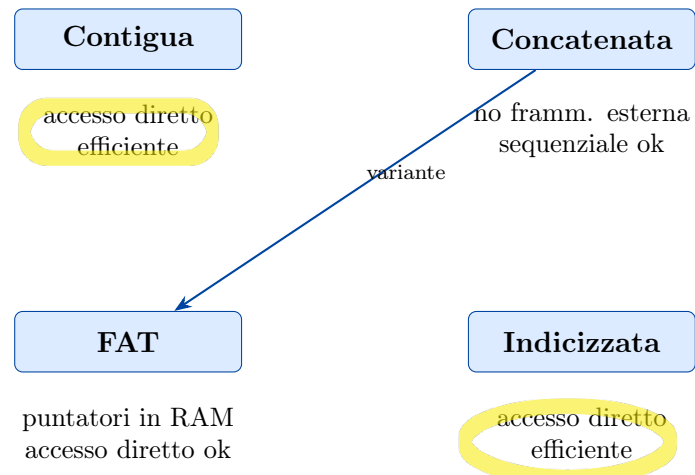
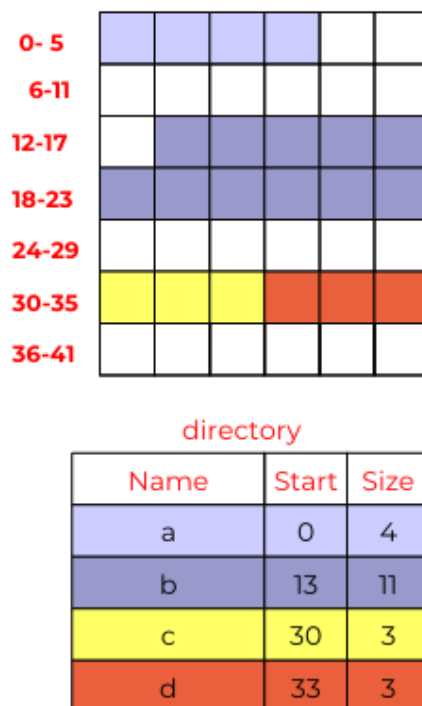


Figura 7.9: Riepilogo dei quattro metodi di allocazione dei blocchi.

1) Allocazione contigua



Nell'allocazione contigua, la entry memorizza il nome del file, il blocco di inizio (Start) e la dimensione (Size).

Figura 7.10: Allocazione contigua: i file occupano sequenze contigue di blocchi. La directory conserva solo il blocco iniziale (Start) e la dimensione (Size). Es: i blocchi del file a vanno da 0 a 5, quelli del b da 12 a 17; ecc

Ogni file è memorizzato in una sequenza **contigua** di blocchi.

Vantaggi:

- Non servono strutture dati per collegare i blocchi.
- Accesso sequenziale efficiente (blocchi contigui = pochi seek).
- Accesso diretto efficiente:

$\text{block} = \text{offset} / \text{blocksize}$, $\text{pos} = \text{offset} \% \text{blocksize}$.

Svantaggi:

- **Frammentazione esterna**: come nella gestione della memoria, si crea nel tempo una serie di buchi non utilizzabili.
- Necessità di politiche di scelta (first fit, best fit, worst fit).
- **I file non possono crescere** (non c'è spazio contiguo garantito dopo il file).

Nota

L'allocazione contigua è ancora usata per supporti ottici (CD, DVD, Blu-ray), dove i file vengono scritti una volta e letti molte volte in modo sequenziale.

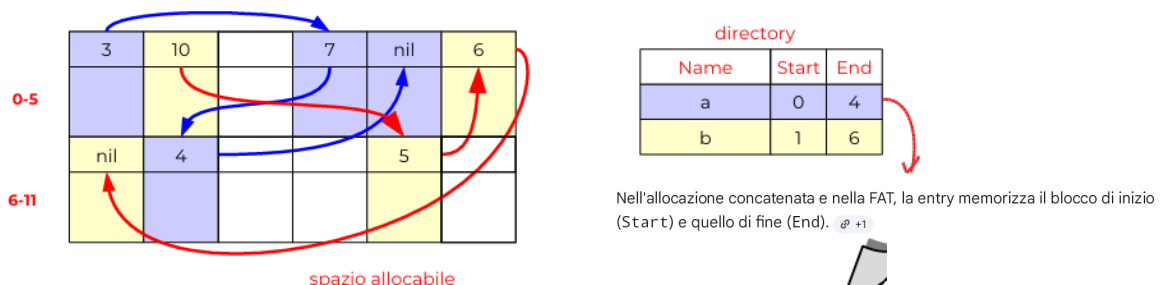
2) Allocazione concatenata

Figura 7.11: Allocazione concatenata: ogni blocco contiene un puntatore al blocco successivo: blocco 0->3; blocco 3->7; ecc.. . Il descrittore conserva puntatore al primo (Start) e all'ultimo blocco (End).

Ogni file è una **lista concatenata** di blocchi. Ogni blocco contiene un puntatore al blocco successivo (indicato dal numero in alto) ; l'ultimo ha **nil**.

Vantaggi:

- Nessuna frammentazione esterna.
- Accesso sequenziale e in *append mode* efficienti.

Svantaggi:

- Accesso diretto inefficiente (occorre seguire la catena).
- Nel tempo i blocchi si disperdono sul disco, aumentando i seek.
- La dimensione utile del blocco non è una potenza di due (una parte è occupata dal puntatore).

Ottimizzazione con i cluster. Per ridurre l'overhead dei puntatori, i blocchi vengono raggruppati in **cluster** (4, 8 o 16 blocchi) allocati come unità indivisibile.

Vantaggio: meno spazio sprecato per puntatori.

Svantaggio: aumenta la frammentazione interna.

3) Allocazione basata su FAT

• Descrizione

- invece di utilizzare parte del blocco dati per contenere il puntatore al blocco successivo si crea una tabella unica con un elemento per blocco (o per cluster)

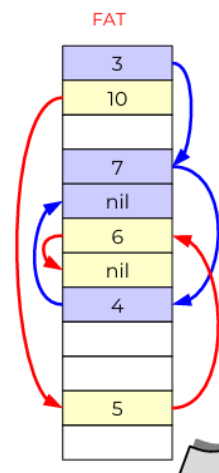
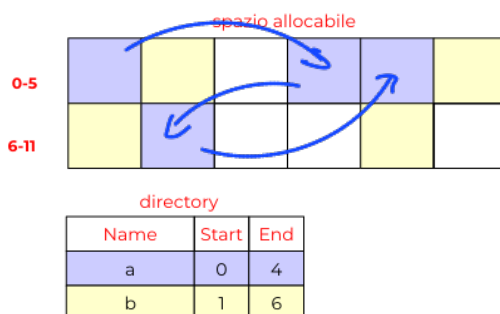


Figura 7.12: Allocazione FAT: invece di tenere i puntatori sparsi nei blocchi dati, si utilizza una tabella centralizzata (File Allocation Table) con un elemento per ogni blocco o cluster.

Definizione – FAT – File Allocation Table

La **FAT** risolve il problema dell'allocazione concatenata estraendo i puntatori dai blocchi dati e raggruppandoli in una **tabella unica** posta all'inizio della partizione. Ogni riga della tabella corrisponde a un blocco fisico sul disco e contiene l'indice del blocco successivo per quel file. Se il blocco è l'ultimo, contiene un valore speciale (EOF o nil).

Vantaggi:

- **Blocchi dedicati interamente ai dati:** al contrario dell'allocazione concatenata, non c'è overhead per i puntatori all'interno dei blocchi dati stessi.
- **Efficienza dell'accesso diretto (tramite caching):** è possibile fare caching in memoria centrale dell'intera FAT (o di parte di essa). In questo modo, l'accesso diretto diventa molto più efficiente, in quanto la lista di puntatori può essere seguita direttamente in RAM senza dover fare lenti accessi al disco.

Svantaggi:

- **Aumento degli accessi al disco per la scansione:** accedere ai dati richiede comunque la lettura preventiva della FAT. Se la tabella non è in memoria, questo raddoppia i passaggi richiesti, aumentando così il numero complessivo di accessi fisici al disco.

Nota

Evoluzione della FAT: Storicamente utilizzata da MS-DOS e Windows 9x, la FAT sopravvive oggi nelle schede di memoria (SD, microSD) e nelle chiavette USB grazie alla sua estrema compatibilità. Le sue varianti (FAT-12, FAT-16, FAT-32, exFAT) differiscono per il numero di bit usati per indirizzare i cluster, il che determina la dimensione massima gestibile per le partizioni e per i singoli file.

4) Allocazione indicizzata

Nell'allocazione indicizzata, la entry memorizza il nome e il puntatore al blocco indice (Index). 

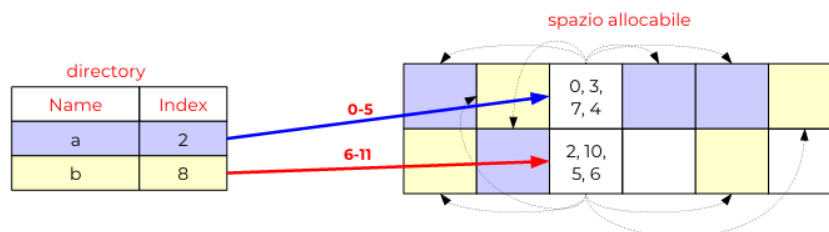


Figura 7.13: Allocazione indicizzata: l'elenco dei blocchi di un file è memorizzato in un *blocco indice* dedicato.

L'elenco dei blocchi che compongono un file viene memorizzato in un **blocco indice** dedicato. Per accedere al file si carica in memoria il suo blocco indice e si usano i puntatori contenuti.

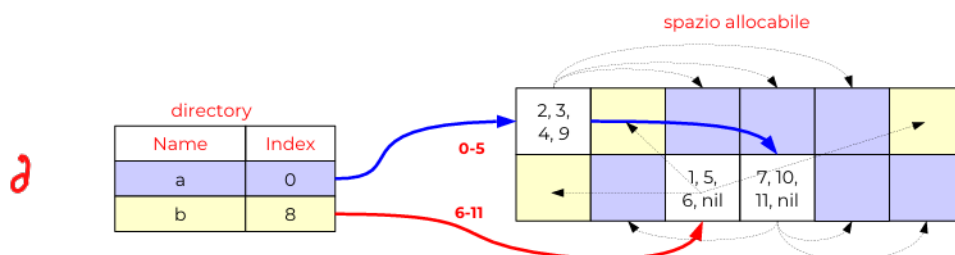
Vantaggi:

- Nessuna frammentazione esterna (come l'allocazione concatenata).
- Efficiente per l'accesso diretto: con il blocco indice in memoria si raggiunge qualunque blocco in un solo accesso al disco.

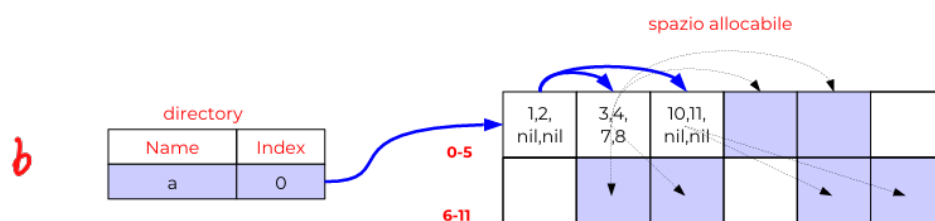
Svantaggi:

- La **dimensione del blocco indice** pone un limite alla dimensione massima del file.
- Blocchi indice troppo grandi $\Rightarrow$ spreco di spazio.
- Blocchi indice troppo piccoli $\Rightarrow$ file piccoli.

4.1) Soluzioni al problema della dimensione del blocco indice.



(a) Concatenazione di blocchi indice: l'ultimo slot punta al blocco indice successivo.



(b) Indice multilivello: un blocco indice di blocchi indice.

Figura 7.14: Due approcci per gestire file di grandi dimensioni nell'allocazione indicizzata.

Il limite principale dell'allocazione indicizzata è che un singolo blocco indice ha una capacità finita. Se un blocco indice può contenere N puntatori, non può gestire file più grandi di $N \times \text{dimensione_blocco}$. Per superare questo limite si usano due strategie:

1. Concatenazione di blocchi indice (Linked Scheme)

In questo schema, il blocco indice non contiene solo puntatori a dati, ma riserva l'ultimo slot per puntare a un secondo blocco indice.

- **Meccanismo:** Si crea una lista concatenata di blocchi indice. Quando il primo è pieno, il sistema ne alloca un altro e lo collega.
- **Il problema:** Sebbene permetta file di dimensioni infinite, distrukge l'accesso diretto per i file molto grandi. Se vuoi leggere l'ultimo blocco di un file enorme, il sistema deve caricare sequenzialmente tutti i blocchi indice della catena per trovare l'indirizzo finale, tornando di fatto ai limiti dell'allocazione concatenata.

2. Indice Multilivello (Hierarchical Scheme)

Si organizza l'indice come un albero. Un blocco indice di "primo livello" non punta ai dati, ma punta ad altri blocchi indice di "secondo livello", i quali puntano finalmente ai dati.

- **Capacità:** La capacità cresce esponenzialmente. Se un blocco contiene 256 puntatori, con 2 livelli possiamo indirizzare $256^2 = 65.536$ blocchi dati.

- **Il problema (Overhead):** Anche per leggere un solo byte, il sistema deve effettuare $N + 1$ letture su disco (dove N è il numero di livelli). Questo rende l'accesso ai file piccoli molto più lento rispetto a un indice singolo.

Nota

Sintesi del Trade-off

- La concatenazione è semplice ma inefficiente per il seek (accesso casuale) in file grandi.
- Il multilivello garantisce un accesso diretto veloce a file enormi, ma penalizza i file piccoli con troppi accessi al disco.

Per questo motivo, UNIX utilizza lo **schema misto** (i-node), che combina puntatori diretti (veloci per file piccoli) e indiretti a più livelli (scalabili per file grandi).

↓
SOLUZIONE a)

↓
SOLUZIONE 4.1) b

Allocazione indicizzata: l'i-node UNIX

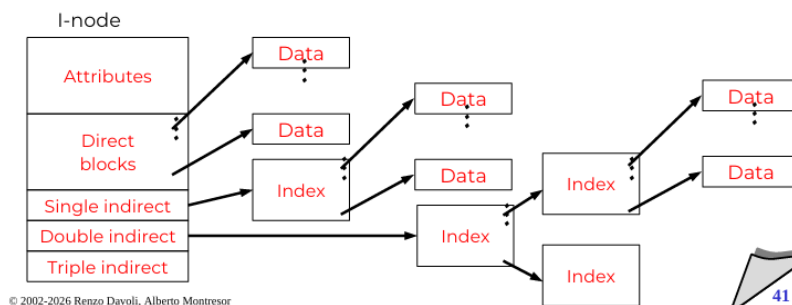


Figura 7.15: Schema misto dell'i-node UNIX: puntatori diretti e puntatori indiretti a uno, due e tre livelli.

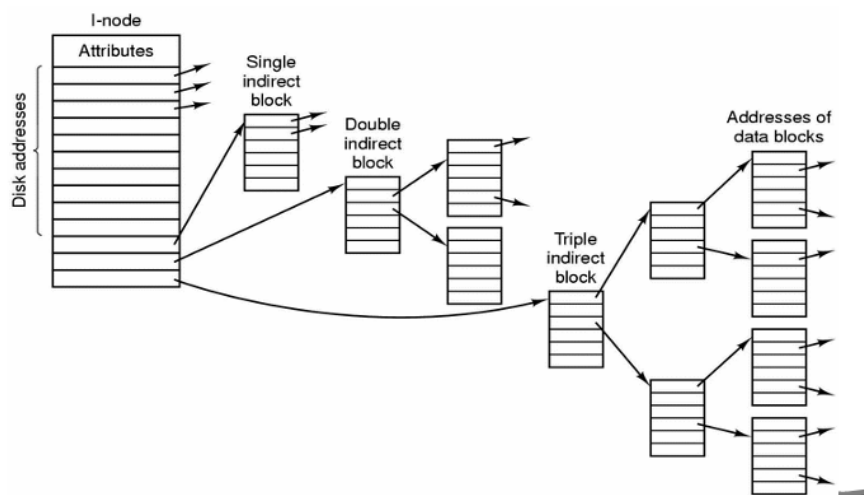


Figura 7.16: Diagramma dettagliato dell'i-node UNIX V7: disk addresses, single/-double/triple indirect block, addresses of data blocks.

UNIX adotta uno **schema misto** molto efficace:

Definizione – i-node (index node)

Struttura dati associata ad ogni file che contiene:

- Gli **attributi** del file (dimensione, permessi, timestamps, ...).
- Un insieme di **puntatori diretti** ai primi blocchi dati (di solito 10–12): efficiente per file piccoli.
- Un puntatore a **blocco indiretto singolo** (single indirect): un blocco contenente puntatori a blocchi dati.
- Un puntatore a **blocco indiretto doppio** (double indirect).
- Un puntatore a **blocco indiretto triplo** (triple indirect).

Nota – Perché lo schema misto è ottimale

File piccoli (la maggioranza) vengono acceduti con 1–2 accessi al disco usando solo i puntatori diretti.

File molto grandi usano i livelli di indirizzione multipla. Il costo extra per i file grandi è giustificato dalla rarità di queste dimensioni nella pratica.

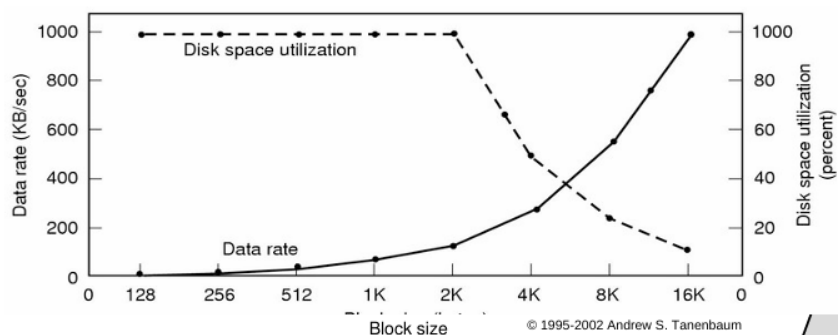
Performance dell'allocazione

Figura 7.17: Relazione tra dimensione del blocco/cluster, data rate e utilizzo del disco. Dati reali su file system: lunghezza mediana dei file ≈ 2 KB.

Il grafico mostra il classico trade-off nella scelta della dimensione del cluster:

- **Cluster grandi:** data rate più alto (meno seek), ma maggiore frammentazione interna (l'ultimo cluster di ogni file è mediamente mezzo vuoto). *→ in quanto blocchi vicini non sono vicini*
- **Cluster piccoli:** minore spreco per frammentazione interna, ma prestazioni più basse.

Il punto di incrocio delle due curve (intorno a 4–8 KB) rappresenta la dimensione di compromesso usata da molti file system moderni (es. ext4 usa tipicamente 4 KB).

Come leggere gli assi

Il trucco per leggere questo grafico è capire che la linea continua e la linea tratteggiata fanno riferimento a due righelli (assi) differenti:

- **Asse orizzontale (in basso) - Block size:** Rappresenta la dimensione del blocco (o cluster) configurata nel file system. Va da blocchi piccolissimi a sinistra (128 byte) a blocchi molto grandi a destra (16 KB). *+1*
- **Asse verticale sinistro - Data rate (KB/sec):** Misura le prestazioni, ovvero la velocità di trasferimento dei dati. I valori (da 0 a 1000) si riferiscono esclusivamente alla linea continua. *+1*
- **Asse verticale destro - Disk space utilization (percent):** Misura l'efficienza, ovvero la percentuale di spazio sul disco utilizzata in modo utile, senza sprechi. I valori in percentuale si riferiscono esclusivamente alla linea tratteggiata. *+1*

7.2.3 Gestione dello spazio libero

Il file system deve tenere traccia dei blocchi liberi per poter allocarne di nuovi (per poter essere dinamico). Esistono tre approcci principali.

1) Mappa di bit (bitmap)

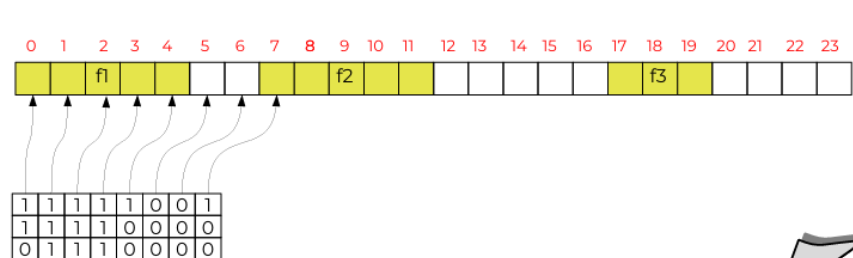


Figura 7.18: Bitmap per la gestione dello spazio libero: ad ogni blocco corrisponde un bit (0 = libero, 1 = occupato).

Ad ogni blocco corrisponde un **bit** in un vettore (*bitmap*).

Bit 0 = blocco libero, bit 1 = blocco occupato.

Vantaggi: Semplice; permette di trovare facilmente aree contigue di blocchi liberi.

Svantaggi: La bitmap può richiedere molto spazio.

Esempio – Calcolo dimensione bitmap

Con blocchi da 1 KB e partizione da 16 GB: 2^{24} blocchi $\Rightarrow 2^{24}$ bit = 2^{21} B = 2 MB per la bitmap.

Nota

Le CPU Intel e Motorola hanno istruzioni hardware per trovare l'offset del primo bit acceso/spento in una word, rendendo la scansione della bitmap molto efficiente.

2) Lista concatenata

• Descrizione

- i blocchi liberi vengono mantenuti in una lista concatenata
- si integra perfettamente con il metodo FAT per l'allocazione delle aree libere

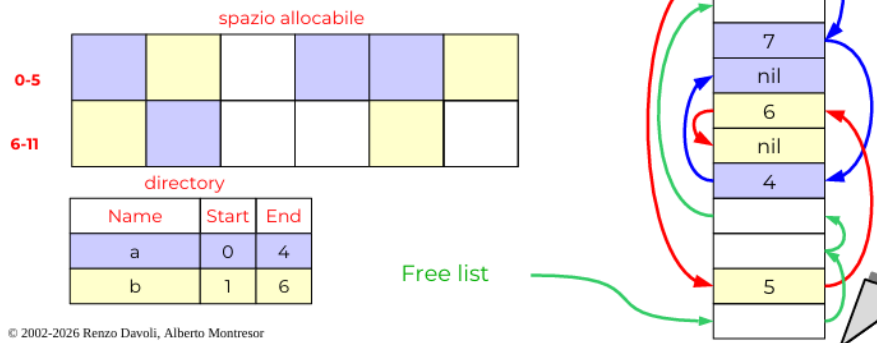


Figura 7.19: Lista concatenata di blocchi liberi, integrata con la FAT.

I blocchi liberi vengono mantenuti in una **lista concatenata**. Si integra perfettamente con il metodo FAT.

Vantaggi: Richiede poco spazio in memoria centrale.

Svantaggi: Allocazione di aree ampie è costosa; trovare aree contigue è molto difficile.

3) Lista concatenata a blocchi

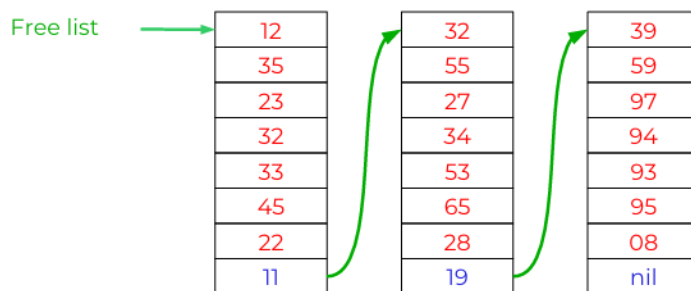


Figura 7.20: Lista concatenata di *blocchi* contenenti puntatori a blocchi liberi. In memoria è sufficiente tenere un solo blocco alla volta.

Variante della lista concatenata: invece di collegare singoli blocchi liberi, si collegano **blocchi di puntatori** a blocchi liberi.

Vantaggi:

- In memoria è sufficiente tenere un solo blocco di puntatori.
- I blocchi contenenti le liste possono essere memorizzati *dentro i blocchi liberi stessi* (nessuna struttura a parte).

Svantaggi: Allocazione di aree contigue ancora difficoltosa.

Confronto numerico dei 3 metodi utilizzati per gestire lo spazio libero:
Con blocchi da 1 KB e partizione da 16 GB:

- Bitmap: $2^{21} \text{ B} \approx 2 \text{ MB}$.
- FAT (puntatori a 3 byte): $2^{24} \times 3 \text{ B} \approx 48 \text{ MB}$.
- Lista a blocchi: spazio occupato *nei blocchi liberi stessi* (costo zero se il disco non è pieno).

L'Obiettivo: Cosa Controllano?

- **Capitolo 5 (Gestione della Memoria Centrale):** Qui le strutture gestiscono la **RAM**. L'obiettivo del Memory Manager è tenere traccia delle celle di memoria per trovare spazio libero in cui caricare i *processi* in esecuzione. [🔗 +4](#)
- **Capitolo 7 (File System):** Qui le strutture gestiscono la **Memoria Secondaria (Disco)**. L'obiettivo del File System è tenere traccia dei blocchi fisici sul disco (hard disk, SSD) per poter allocare nuovo spazio e salvare i *file*. [🔗 +2](#)

La Posizione: Dove sono Memorizzate?

- **Capitolo 5 (RAM):** La Mappa di Bit e la Lista con Puntatori per la gestione della memoria centrale sono memorizzate interamente nella **RAM stessa**, all'interno di uno spazio speciale riservato in modo stabile al Kernel del sistema operativo. [🔗](#)
- **Capitolo 7 (Disco):** Le strutture per lo spazio libero del file system sono memorizzate in modo permanente **sul disco**, all'interno della singola partizione formattata (nello specifico, l'area di "Gestione spazio libero" posizionata subito dopo il Superblock). Per garantire prestazioni accettabili, quando il computer è acceso queste porzioni di disco vengono lette e mantenute temporaneamente in RAM (nella *block cache*), ma la loro natura è persistente.

Caratteristica	Capitolo 5 (RAM)	Capitolo 7 (Disco / File System)
Cosa gestisce?	Memoria Centrale (RAM)	Memoria Secondaria (Disco)
Unità di allocazione	Cella o piccolo blocco di RAM	Blocco o cluster del disco
Dove risiede?	RAM (Spazio del Kernel)	Disco (Strutture della partizione)

Analisi delle Strutture e Differenze

La Mappa di Bit (Bitmap) Il concetto di base è identico: la memoria viene divisa in unità logiche e ad ogni unità corrisponde un singolo bit (0 se libera, 1 se occupata). Il grande vantaggio in entrambi i casi è la semplicità con cui si possono trovare aree contigue cercando visivamente una sequenza di zeri consecutivi.

- **Nella RAM (Capitolo 5):** L'unità di misura può essere molto piccola (es. 4 byte o 1 KB). Cercare k unità consecutive libere richiede di scansionare la bitmap, che è un'operazione lenta con costo computazionale $O(m)$ (dove m è il numero totale di unità). [🔗 +2](#)
- **Nel Disco (Capitolo 7):** L'unità di misura è tipicamente il blocco del disco. Dato che i dischi moderni sono enormi (es. 16 GB), la bitmap diventa colossale e arriva a occupare megabyte interi (es. 2 MB) di spazio prezioso. Per fortuna, a livello hardware, alcune CPU offrono istruzioni specifiche per trovare velocemente il primo bit acceso/spento in queste mappe, rendendo la scansione molto più rapida. [🔗 +3](#)

In entrambi gli ecosistemi, il sistema operativo deve affrontare lo stesso identico compromesso: se scegli unità di allocazione molto grandi avrai una bitmap piccolissima, ma sprecherai tantissimo spazio all'interno dei blocchi (frammentazione interna); se scegli unità piccole, ottimizzi lo spazio ma la tua bitmap diventerà gigantesca. [🔗](#)

La Lista Concatenata Qui le differenze implementative si fanno molto più marcate, perché la natura hardware del disco e della RAM impone scelte diverse.

- **Nella RAM (Capitolo 5):** Il sistema utilizza una *lista doppiamente collegata* i cui nodi sono vere e proprie strutture dati salvate nello spazio del Kernel. Ogni singolo nodo della lista non contiene i dati dei programmi, ma solo metadati: indica se quell'area è un processo (P) o un "buco" libero (H), l'indirizzo di inizio, la dimensione e i puntatori di navigazione. Grazie a questa struttura, quando un processo termina, unire il suo spazio liberato con un buco adiacente è un'operazione istantanea con costo $O(1)$. [🔗 +3](#)
- **Nel Disco (Capitolo 7):** Sul disco, l'approccio è diverso. Invece di creare nodi separati ricchi di metadati, i puntatori che legano i blocchi liberi vengono scritti direttamente all'interno dello spazio dei blocchi liberi stessi (a costo zero se il disco non è pieno), oppure la lista si integra perfettamente all'interno di una tabella centralizzata come la FAT. Inoltre, a differenza della RAM, allocare aree grandi e contigue sul disco seguendo una lista è estremamente difficoltoso e lento.

7.2.4 Implementazione delle directory

Una directory è un file speciale suddiviso in **directory entry**. Ogni entry deve permettere di accedere a tutte le informazioni del file: nome, attributi, informazioni di allocazione.

Dove si memorizzano gli attributi?

nome file
attributi
info allocaz.
nome file
attributi
info allocaz.

nome	i-node
nome	i-node
nome	i-node
nome	i-node

Figura 7.21: in alto (MS-DOS): la directory entry contiene direttamente tutti gli attributi del file. E' quindi un elenco di descrittore di file. Sotto (UNIX): la directory entry contiene solo il nome e l'indice di i-node; gli attributi sono nell'i-node.

- **Attributi nella directory entry (MS-DOS):** tutta l'informazione è in un posto solo.
- **Attributi nell'i-node (UNIX):** la directory entry contiene solo (nome, numero_inode). Gli attributi sono nell'i-node. Questo permette gli hard link senza duplicare le informazioni.

Problema della lunghezza dei nomi

La memorizzazione di nomi a lunghezza variabile nelle directory entry comporta un trade-off:

- **Lunghezza fissa:** soluzione semplice, ma spreca spazio se la dimensione riservata è troppo grande, oppure limita i nomi se è troppo piccola.
- **Lunghezza variabile:** la struttura dati diventa più complessa. Due approcci possibili:

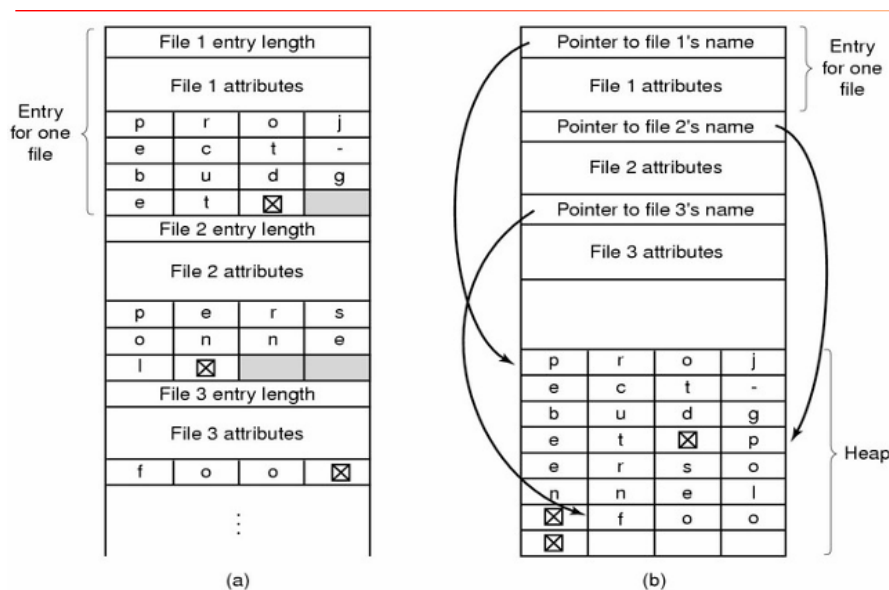


Figura 7.22

1. **Lunghezza variabile (Accodamento):** Ogni entry ha una lunghezza variabile dichiarata esplicitamente, e il nome del file viene scritto subito dopo gli attributi. Le entry vengono poi salvate sul disco accodate l'una all'altra.
2. **Lunghezza variabile (Heap):** Si mantiene una directory entry a lunghezza fissa per comodità di lettura, ma al posto del nome si inserisce un puntatore. Questo puntatore rimanda a un'area di memoria separata, chiamata *heap*, dove viene effettivamente memorizzato il testo del nome del file.

Strutture Dati per la Ricerca

Quando un utente cerca un file, il sistema operativo deve scorrere la directory per trovare la *directory entry* corrispondente. Vi sono due approcci principali per organizzare e cercare queste entry:

- **Lista Lineare (Array)**

- **Descrizione:** Le directory entry vengono semplicemente accodate l'una dopo l'altra in una lista sequenziale.
- **Pro:** È la struttura più semplice in assoluto da implementare.
- **Contro:** Diventa gravemente inefficiente nel caso di directory di grandi dimensioni. Per trovare un file, il sistema deve potenzialmente leggere l'intera lista dall'inizio alla fine.

- **Tabella Hash**

- **Descrizione:** Si calcola una funzione di hash partendo dal nome del file, che restituisce direttamente un puntatore alla directory entry associata.
- **Pro:** I tempi di ricerca sono drasticamente ridotti, indipendentemente dal numero di file presenti nella directory.
- **Contro:** Introduce maggiore complessità architetturale. Occorre stabilire a priori la dimensione della tabella e implementare un metodo per la gestione delle collisioni (quando due nomi generano lo stesso hash). Se ci sono numerose collisioni, l'efficienza degrada.

7.2.5 Performance del file system

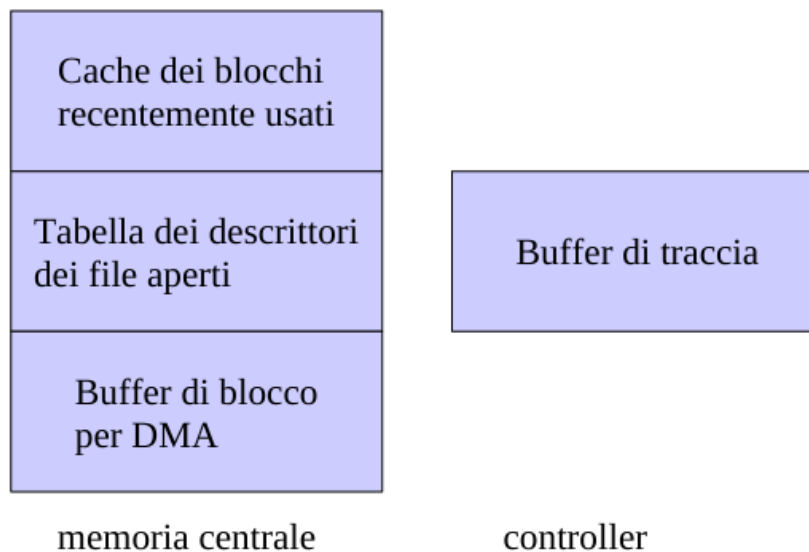


Figura 7.23

Le principali tecniche di ottimizzazione sono:

- **Block cache** (buffer cache): i blocchi letti di recente vengono mantenuti in memoria. Se richiesti nuovamente si servono dalla cache senza accedere al disco.
- **Read-ahead**: si prelevano in anticipo i blocchi sequenzialmente successivi, anticipando il comportamento dell'accesso sequenziale.
- **Tabella dei file aperti**: struttura in memoria che descrive ogni file aperto, evitando di risolvere il percorso ad ogni accesso.
- **Buffer per DMA**: il trasferimento avviene direttamente tra controller del disco e memoria, senza coinvolgere la CPU per ogni byte.

Nota – Osservazione sulla block cache

La block cache è particolarmente vantaggiosa per le strutture del file system accedute frequentemente: FAT, bitmap, i-node e directory. Nel caso dell'allocazione basata su FAT, ad esempio, tenere l'intera tabella in memoria rende l'accesso diretto molto più efficiente, eliminando la necessità di leggere la FAT dal disco ad ogni attraversamento della lista di puntatori.

Attenzione – Coerenza e caching

Il caching introduce un rischio: i dati in memoria possono non essere ancora stati scritti su disco. In caso di interruzione di corrente o crash, le zone più critiche del file system (FAT, bitmap, i-node, directory) possono risultare inconsistenti. Questo problema viene affrontato con le tecniche descritte nella sezione successiva.

7.2.6 Tecniche per garantire la coerenza

Il caching può causare **inconsistenze** nel file system: se il sistema viene spento bruscamente (es. mancanza di corrente) con blocchi modificati ancora in cache, il disco rimane in uno stato incoerente. Il problema è particolarmente critico per FAT, bitmap, i-node e directory.

Esistono due filosofie:

1. **curare** (rilevare e correggere le inconsistenze a posteriori: fsck o scandisk);
2. **prevenire** (evitarle tramite transazioni, ovvero tecniche (journaling file system : ext3, reiserfs).

1) Curare:

fsck (file system check) è il checker UNIX; **scandisk** è l'equivalente Windows. Le fasi di **fsck**:

1. **Phase 1 – Check Blocks and Sizes**: scandisce la tabella degli i-node cercando incoerenze.
2. **Phase 2 – Check Path-Names**: verifica che le directory puntino a i-node legali.
3. **Phase 3 – Check Connectivity**: scandisce l'albero per verificare che tutti i file siano raggiungibili (*Lost+Found*).
4. **Phase 4 – Check Reference Counts**: verifica il numero di riferimenti ad ogni file (contatore negli i-node).
5. **Phase 5 – Check Cylinder Group**: verifica la coerenza tra i-node e bitmap dei blocchi liberi/occupati.
6. **Phase 6 – Salvage Cylinder Groups**: aggiorna le tabelle per salvare i cambiamenti effettuati.

Attenzione – Limiti di fsck

fsck è un'operazione lenta (scansiona tutto il disco) e non garantisce il ripristino di tutti i dati. Su dischi di grandi dimensioni può richiedere molti minuti o ore.

2) Prevenire: File system journaling (basati su log)

Definizione – Journaling File System

Un file system **journaling** tratta ogni aggiornamento come una **transazione**: un'operazione atomica (“tutto o niente”). Le transazioni vengono prima scritte in un'area speciale del disco chiamata **log** (o *journal*), poi applicate al file system.

Funzionamento del log.

1. Una transazione è considerata **committed** quando è stata scritta nel log, anche se il file system non è ancora aggiornato.
2. Periodicamente, le transazioni nel log vengono **applicate** al file system (operazione di *checkpoint*).
3. Dopo l'applicazione, la transazione viene rimossa dal log.
4. In caso di crash, al riavvio le transazioni *nel log ma non ancora applicate* vengono **ripetute** (operazione di *redo*).

Nota – Esempi di file system journaling

- **ext3, ext4**: estensioni journaling di ext2 (Linux).
- **reiserfs**: file system journaling per Linux.
- **NTFS**: file system journaling di Windows NT/XP/7/...
- **HFS+**: Apple (con journaling opzionale).
- **ZFS, Btrfs**: file system moderni con meccanismi di integrità ancora più avanzati (copy-on-write).

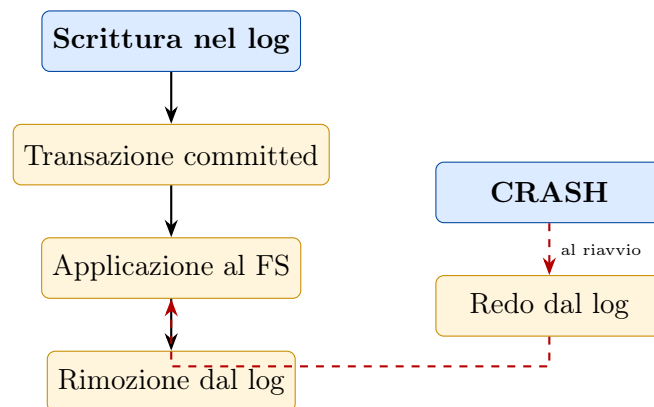


Figura 7.24: Ciclo di vita di una transazione in un journaling file system. In caso di crash, le transazioni committed ma non ancora applicate vengono ripetute dal log.

7.2.7 Esempi di file system reali

MS-DOS: FAT

- **Nomi file:**
 - 8+3 caratteri
 - indici di blocco a 12-16 bit (MS-DOS, W95), 32 bit (W98)

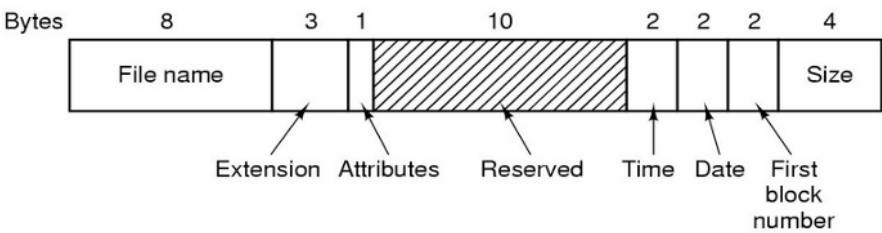


Figura 7.25: Struttura di una directory entry MS-DOS: 32 byte totali. Il campo “File name” occupa 8 byte, l’estensione 3 byte.

La directory entry di MS-DOS è una struttura fissa di 32 byte:

Campo	Contenuto
File name	8 byte (nome base).
Extension	3 byte (estensione).
Attributes	1 byte (sistema, nascosto, archivio, ...).
Reserved	10 byte (data di creazione, etc.).
Time	2 byte (ora ultima scrittura).
Date	2 byte (data ultima scrittura).
First block	2/4 byte (primo cluster nella FAT).
Size	4 byte (dimensione in byte).

Dimensione blocco/cluster	FAT-12	FAT-16	FAT-32
0.5 KB	2 MB		
1 KB	4 MB		
2 KB	8MB	128 MB	
4 KB	16MB	256 MB	1 TB
8 KB		512 MB	2 TB
16 KB		1024 MB	2 TB
32 KB		2048 MB	2 TB

Figura 7.26: Dimensioni massime delle partizioni in funzione del tipo di FAT e della dimensione del cluster.

Varianti FAT-12, FAT-16, FAT-32. La variante FAT-32 (con indici a 32 bit per cluster) è quella più diffusa oggi per supporti rimovibili (schede SD, chiavette USB). Con cluster da 4 KB gestisce partizioni fino a 1 TB.

Windows 9x: directory entry estesa

Struttura di una directory entry "compatibile con MSDOS"

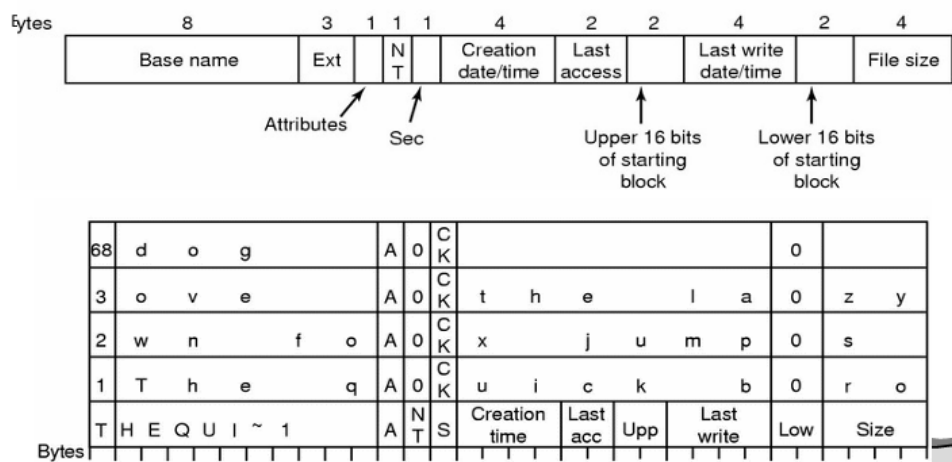


Figura 7.27: Directory entry Windows 9x: compatibile con MS-DOS ma con campi aggiuntivi per data di creazione, ultimo accesso e ora dell'ultimo accesso.

Windows 9x estende la directory entry MS-DOS aggiungendo:

- Data e ora di **creazione** del file.
- Data dell'**ultimo accesso**.
- I 16 bit superiori e inferiori del numero del primo cluster (necessari per FAT-32).

Per supportare i nomi lunghi (LFN), Windows 9x usa entry aggiuntive "LFN" precedenti all'entry normale (il flag NT indica le entry LFN).

UNIX V7: i-node e directory lookup

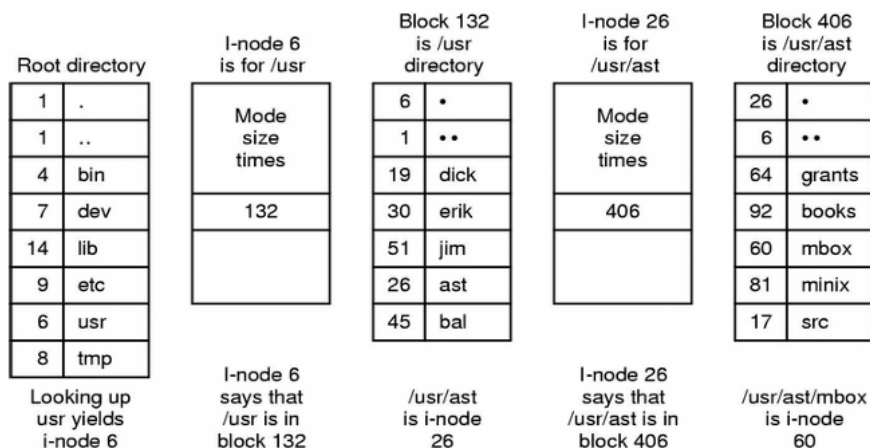


Figura 7.28: Risoluzione del percorso `/usr/ast/mbox` in UNIX V7. La root directory mappa `usr` → i-node 6; l'i-node 6 dice che `/usr` è nel block 132; e così via fino al file finale.

Il processo di **path resolution** in UNIX segue questi passi:

1. La **root directory** viene caricata. Cerca l'entry `usr` ⇒ i-node 6.
2. L'i-node 6 dice che la directory `/usr` è nel blocco 132.
3. Il blocco 132 (directory `/usr`) contiene l'entry `ast` ⇒ i-node 26.
4. L'i-node 26 dice che `/usr/ast` è nel blocco 406.
5. Il blocco 406 contiene l'entry `mbox` ⇒ i-node 60.
6. L'i-node 60 contiene gli attributi e i puntatori ai dati di `mbox`.

Nota – Efficienza della risoluzione del percorso

In UNIX ogni componente del percorso richiede tipicamente 2 accessi al disco (lettura dell'i-node + lettura del blocco directory). Un percorso con n componenti richiede $\approx 2n$ accessi. La cache degli i-node e delle directory riduce drasticamente il numero di accessi effettivi.

7.3 Riepilogo comparativo

	Contigua	Concatenata	FAT	Indicizzata
Accesso seq.	***	***	***	***
Accesso dir.	***	*	**	***
Framm. esterna	*	***	***	***
File crescente	*	***	***	***
Uso S.O.	CD/DVD	(base)	DOS/FAT	UNIX/ext

*** = ottimo, ** = buono, * = scarso

	Bitmap	Lista concat.	Lista a blocchi
Spazio in RAM	medio (2MB/16GB)	poco	poco (1 blocco)
Aree contigue	facile	difficile	difficile
Spazio su disco	fisso	nei blocchi lib.	nei blocchi lib.
Uso tipico	ext, NTFS	FAT	UFS/FFS

Le Due Applicazioni a Confronto

La differenza fondamentale sta nel **dove** si trova lo spazio da gestire e **chi** lo sta gestendo:

Caratteristica	Capitolo 5: Memoria Centrale (RAM)	Capitolo 7: File System (Disco/Memoria Secondaria)
Cosa gestisce?	La memoria RAM principale del computer.	Lo spazio sul disco rigido (o SSD, chiavetta USB, ecc.).
Come si chiama il "pezzetto"	Unità di allocazione.	Blocco (o blocco dati).
Scopo principale	Trovare spazio libero per caricare i processi in RAM.	Trovare spazio libero per salvare i file sul disco.

Pro e Contro Condivisi

Poiché la struttura è la stessa, anche i vantaggi e gli svantaggi che il Sistema Operativo deve affrontare sono molto simili in entrambi i mondi:

- **Vantaggio:** È una struttura molto semplice che permette di trovare facilmente aree contigue di spazio libero (cercando una sequenza di zeri consecutivi). 
- **Svantaggio:** La bitmap stessa occupa spazio. Nel caso del File System, ad esempio, per gestire un disco da 16 GB con blocchi da 1 KB, la bitmap occuperà ben 2 MB di spazio. Nel caso della RAM, cercare blocchi contigui richiede tempo (costo $O(m)$ dove m è il numero totale di unità).  +3
- **Il Trade-off delle dimensioni:** In entrambi i casi, se si scelgono "pezzetti" molto piccoli si avrà una bitmap gigantesca (ma poca frammentazione interna); se si scelgono pezzetti grandi si avrà una bitmap compatta, ma molto spazio sprecato all'interno dei blocchi (frammentazione interna).

Ecco le differenze principali spiegate in modo chiaro:

1. L'approccio di UNIX (Struttura snella e i-node)

In UNIX, si usa il meccanismo degli i-node, il che significa che le directory sono concepite per essere estremamente leggere.

- **Directory leggerissime:** Una directory entry in UNIX contiene esclusivamente due informazioni: il nome del file e il puntatore al suo i-node. Tutti gli attributi del file (dimensione, permessi, data) sono conservati altrove, nell'i-node stesso.
S_O___07__Sistemi_Op... +2
- **Risoluzione del percorso (Path Resolution):** Per trovare un file (es. `/usr/ast/mbox`), il sistema fa una sorta di "ping-pong". Legge l'i-node della radice, che lo punta al blocco della cartella `usr`; legge la cartella `usr` per trovare l'i-node di `ast`; legge l'i-node di `ast`, e così via.
S_O___07__Sistemi_Op... +1
- **I costi teorici e pratici:** Teoricamente, ogni pezzo del percorso richiede 2 accessi al disco (lettura dell'i-node + lettura del blocco directory). Un percorso di n componenti richiederebbe quindi $2n$ accessi. Tuttavia, nella realtà è **molto efficiente** perché il sistema operativo mantiene una *cache degli i-node e delle directory* in RAM, abbattendo drasticamente gli accessi fisici al disco.
S_O___07__Sistemi_Op... +2

2. L'approccio MS-DOS / Windows 9x (FAT e Struttura pesante)

In MS-DOS e nelle vecchie versioni di Windows, il file system FAT adotta un approccio in cui tutto è raggruppato insieme.

- **Directory "pesanti":** La directory entry non contiene solo il nome, ma integra al suo interno tutti gli attributi del file (dimensione, data, ora, attributi di sistema e il puntatore al primo blocco dati) in una struttura fissa di 32 byte. Nelle versioni per Windows 9x, per supportare i nomi lunghi, vengono aggiunte ulteriori entry accodate l'una all'altra, appesantendo ulteriormente la struttura.
S_O___07__Sistemi_Op... +3
- **La ricerca lineare:** I file dentro una directory vengono salvati come una "Lista Lineare" (un semplice array dove le entry sono accodate una dopo l'altra).
S_O___07__Sistemi_Op...
- **Il collo di bottiglia:** Questo approccio è la rovina dell'efficienza per le cartelle grandi. Per trovare un file, il sistema deve potenzialmente scorrere e leggere l'intera lista dall'inizio alla fine. L'unica vera ancora di salvezza per le performance in accesso diretto di questo file system è caricare e mantenere l'intera tabella FAT nella memoria centrale (RAM) per evitare continui accessi lenti al disco.
S_O___07__Sistemi_Op... +1

Il verdetto sull'efficienza

UNIX scala molto meglio. Anche se l'indirizzamento degli i-node sembra richiedere più "salti" teorici tra i blocchi, avere directory entry che contengono solo un nome e un numero rende le cartelle piccolissime in termini di byte. Questo permette di scansionarle velocemente e di tenerle facilmente nella RAM (block cache).

S_O___07__Sistemi_Op... +1

MS-DOS/FAT, mettendo tutti gli attributi direttamente dentro la directory, aveva senso sui vecchi dischi o floppy per avere "tutto a portata di mano" leggendo un solo blocco, ma diventa gravemente inefficiente quando le directory si riempiono di migliaia di file che devono essere letti in sequenza dall'inizio alla fine.